Engineering Personality in Large Language Models: A Comprehensive Evaluation of Injection Methodologies for Massive Multi-Agent Simulations

Executive Summary

For the deployment of a scalable, multi-agent social simulation requiring millions of distinct personas driven by 28 continuous psychometric facets, relying exclusively on prompting or full fine-tuning is computationally and behaviorally suboptimal. Prompting degrades foundational reasoning capabilities and fails to maintain high-fidelity continuous traits over long conversational horizons. Conversely, per-agent fine-tuning is architecturally impossible at the scale of millions due to volatile memory constraints. The recommended production architecture is a tightly integrated hybrid pipeline: (1) a baseline model fine-tuned via Low-Rank Adaptation (LoRA) to establish 10–20 stable macro-archetypes; (2) activation steering (representation engineering) injected dynamically at the residual stream level during inference to modulate the 28 continuous Big Five-derived facets at near-zero marginal latency; and (3) lightweight, prefix-cached system prompting to inject agent-specific explicit if-then rules, names, and graph-retrieved memory context. This recommendation shifts strictly if scaling activation steering across 28 simultaneous vectors proves excessively non-orthogonal in frontier models, at which point a Mixture-of-LoRA-Experts (MoE) approach served via S-LoRA becomes the mandatory, albeit more compute-intensive, alternative.

1. Prompting (No Weight Changes)

Prompting modifies a Large Language Model's (LLM) behavior at inference time by prepending context. While it requires no parameter updates and is trivial to implement, it exhibits significant behavioral drift, induces documented capability costs, and struggles to render deeply nuanced continuous psychometric profiles.

a. Reliability and Scaling of Prompted Traits

(Strongly Supported): LLMs manifest internal structures analogous to human personality dimensions when properly prompted. Psychometric evaluations utilizing adapted human instruments—such as the 300-item IPIP-NEO and the 44-item Big Five Inventory (BFI)—demonstrate that large, instruction-tuned models reliably express Big Five traits[1]. To measure this, researchers utilize structured prompting methodologies that simulate demographic and personality variables, evaluating the model by log-probability assignment over discrete response options (e.g., Likert scales) to minimize item-order contamination[2]. Reliability, quantified by standard psychometric metrics including Cronbach's α, Guttman's λ_6, and McDonald's ω, frequently exceeds 0.90 in frontier instruction-tuned models[2].

(Strongly Supported): The reliability of trait expression scales positively with model size and instruction tuning. Base models (e.g., untuned 62B parameter architectures) yield low internal consistency and highly unstable psychometric profiles[2]. Instruction tuning fundamentally organizes the model's latent space, allowing it to adhere to a persistent internal narrative. Furthermore, human evaluators can perceive these prompted personality traits in generated text with up to 80% accuracy, though this perceptual accuracy drops significantly when evaluators are informed of the AI authorship, indicating a human bias in interpreting synthetic personas[3].

b. Persona Drift and Long-Context Mitigation

(Strongly Supported): Over long, multi-turn dialogues, prompted personas suffer from "persona drift," where the model gradually sheds its assigned traits and collapses back to its default instruction-tuned "helpful assistant" persona[4].

(Mixed): Because the persona exists solely in the KV cache and not in the model weights, high context utilization frequently dilutes the attention allocated to the original system instructions[5]. Mitigations include continuous re-injection of the persona prompt, dynamic summarization of traits injected into the context window, and periodic reminders inserted directly into the user turn. While effective, these methods exacerbate KV cache bloat, significantly increasing the time-to-first-token (TTFT) and degrading the economic viability of serving millions of agents.

c. Capability Cost: Factual and Reasoning Degradation

(Strongly Supported): Persona prompting actively degrades a model's foundational reasoning and factual retrieval capabilities. Research from the PRISM study at USC and the "Playing Pretend" study by Wharton Generative AI Labs demonstrates that injecting an "expert" persona shifts the model's objective from factual accuracy to stylistic imitation[6].

When evaluated on the MMLU benchmark, deploying a standard expert persona drops overall accuracy from a baseline of 71.6% to 68.0%, and down to 66.3% with highly verbose persona prompts[6]. The degradation is heavily concentrated in zero-shot logical reasoning, mathematics, and coding. For example, on specific MT-Bench math evaluations, adding a "math expert" persona caused scores to plummet from 9/10 to 1.5/10[6]. The underlying mechanism is that confident, authoritative language styles induced by the persona compete directly with precise factual retrieval pathways established during pre-training. Low-knowledge personas (e.g., "Toddler" or "Layperson") similarly harm performance, often triggering refusal behaviors in models like Gemini 2.5 Flash when out-of-domain questions are asked[9].

d. Numeric Scores vs. Natural-Language Descriptors vs. If-Then Rules

(Strongly Supported): LLMs respond significantly better to natural-language descriptors than to raw numeric facet scores[2]. In prompt-based shaping, utilizing linguistic qualifiers (e.g., "extremely extraverted," "somewhat introverted") derived from Goldberg's 104 trait adjectives aligns accurately with intended outputs, achieving a Spearman's $\rho \geq 0.90$ between ordinal targets and realized trait medians[2]. Numeric scores fail because autoregressive models lack deep semantic grounding for arbitrary continuous scales without explicit text translation. If-then rules (e.g., "defensive when criticized") operate via in-context logic rather than deep persona simulation. They are highly effective for conditional behavioral triggers but insufficient for

embodying continuous, passive trait expressions.

e. Known Failure Modes

(Strongly Supported): The most prominent failure modes of pure prompting include:

1. **Caricature and Stereotyping**: Models exaggerate assigned traits, collapsing multidimensional nuance into one-dimensional stereotypes (e.g., extreme neuroticism resulting in incessant panic rather than subtle anxiety)[4].
2. **Social-Desirability and Positivity Bias**: Standard alignment protocols (RLHF/DPO) heavily bias models toward agreeableness and high conscientiousness[10]. Attempting to prompt dark-triad, highly disagreeable, or genuinely hostile traits often fails, as the underlying safety training overrides the prompt, causing the model to apologize or revert to a helpful tone.
3. **Mode Collapse Across Agents**: When simulating millions of agents using only prompts, a lack of distinctiveness emerges. Because the underlying weight distribution remains identical, agents begin to sound structurally and syntactically alike—exhibiting the same sentence lengths, vocabulary distributions, and pacing[5].

2. Fine-Tuning: Embedding Personality in Weights

Fine-tuning alters the parameters of the base model, embedding behavioral phenotypes directly into the weights. This mitigates the capability degradation associated with verbose context prompts and dramatically increases the fidelity of trait expression.

a. Methodologies of Key Architectures

(Strongly Supported): The literature reveals three primary methodologies for personality fine-tuning:

- **Character-LLM (Shao et al., 2023)**: This framework operates by transforming character profiles into first-person datasets. It begins with "Experience Reconstruction," where an LLM generates detailed scenes, internal thoughts, and emotional responses based on a character profile. This is followed by "Experience Upload," where the base model undergoes supervised fine-tuning (SFT) on these reconstructed experiences[11]. To prevent anachronistic hallucinations, the authors introduce "Protective Experiences"—synthetic data that explicitly trains the model to express ignorance regarding concepts outside its temporal or contextual domain (e.g., training a Beethoven agent to act bewildered if asked about Python programming)[11]. The training setup utilizes 10 epochs with an AdamW optimizer, a batch size of 64, and a maximum context length of 2048 tokens.
- **P-Tailor / P-React (Dan et al., 2024)**: To handle multiple traits without catastrophic interference, P-Tailor utilizes a Mixture of LoRA Experts (MoE) architecture[5]. It splits a single LoRA module into N experts ($\Delta W = \sum_{i=1}^{N} \omega_{ij} B_i A_i$), where ω_{ij} is generated by a personality-guided routing module. P-Tailor curates a high-quality Personality Crafting Dataset based on personality theories and employs a novel "Personality Specialization Loss" (PSL). This loss forces individual LoRA experts to specialize in distinct traits, enhancing personality simulation by minimizing cross-trait contamination[5].

- **Orca (Huang, 2024)**: Orca integrates personality traits through a multi-stage pipeline utilizing psychometric modeling and trajectory tuning[12]. Data synthesis heavily leverages frameworks like the Personality Database (PDB) or synthetically generated Q&A pairs (e.g., PersonaChat, OCEAN-Chat) to establish conditional role-play[14]. It focuses on enhancing the role-playing abilities of LLMs by strictly integrating continuous personality dimensions into the training data loop[15].

b. Data and Compute Scaling

(Mixed): The volume of data required for robust trait expression is surprisingly low for single archetypes—often 1,000 to 5,000 highly curated conversational pairs—but scales drastically for multidimensional continuous traits[5]. Compute requirements scale linearly with model size for Full SFT, rendering it economically unviable for agent simulation. However, Parameter-Efficient Fine-Tuning (PEFT) methods like LoRA allow rapid adaptation. A standard configuration (e.g., rank $r = 32$, $\alpha = 64$, learning rate $1\text{e-}5$) can successfully fine-tune a 7B–8B parameter model on a single A100 GPU in a matter of hours[16]. Preference optimization methods (RLHF, PPO, DPO) yield stronger alignment to specific psychometric outputs but require expensive reward model training and substantially more compute than standard SFT[17].

c. Continuous Trait Handling

(Contested): While researchers frequently claim continuous trait support, empirical demonstrations largely focus on binary extremes (High vs. Low). P-Tailor, for example, categorizes the Big Five into high and low levels, resulting in 10 distinct structural traits rather than a true continuous 5D or 28D spectrum[5]. Achieving 28 continuous dimensions strictly through fine-tuning requires either an unmanageable number of specific training datasets or the mathematical interpolation of LoRA weights post-training. Interpolating weights for highly complex psychological facets currently lacks rigorous validation and frequently incurs representation collapse, where the model outputs become incoherent.

d. Catastrophic Forgetting and Capability Loss

(Strongly Supported): Overfitting is intentionally weaponized in character simulation to embed the persona deeply and override the default assistant tone[11]. However, this leads to catastrophic forgetting of general reasoning capabilities. If a model is fine-tuned strictly on conversational data exhibiting high neuroticism, it loses its ability to perform objective logic or mathematical reasoning. Mitigation requires mixing character data with a replay buffer of general instruction-tuning data (maintaining a 10-20% ratio of general knowledge tasks) or utilizing isolated LoRA adapters that leave the base model weights completely intact[5].

e. Multi-Persona Serving Architecture

(Strongly Supported): Serving one distinct model per archetype is economically unfeasible for simulations involving millions of agents. The literature universally supports a single "profile-conditioned" base model serving multiple personalities dynamically. This is achieved via Multi-LoRA inference—where discrete LoRAs represent broad archetypes (e.g., "Introverted Academic")—combined with input conditioning (prompts or vectors) to shape the specific, unique nuances of the individual agent[5].

3. Activation Steering: Representation Engineering

Activation steering modulates behavior by algebraically modifying the residual stream at inference time. It operates directly in the model's representation space without altering weights or expanding the KV cache, offering a powerful mechanism for continuous trait control.

a. The Persona Vectors Pipeline

(Strongly Supported): Developed by Chen, Arditi, Sleight, Evans, and Lindsey (2025; repo: safety-research/persona_vectors), the Persona Vectors methodology extracts behavioral directions by mapping natural-language descriptions into latent space[16].

1. **Extraction via Contrastive Prompts**: An LLM (e.g., Claude-3.7-Sonnet) generates pairs of trait-promoting (positive) and trait-suppressing (negative/antonym) system prompts, alongside a set of evaluation questions[16].
2. **Activation Harvesting**: The target model processes both positive and negative prompts. The pipeline records the residual-stream activations, typically averaged over the response tokens at specific intermediate layers (e.g., layer 20 in a 7B model)[16]. Intermediate layers are targeted because early layers capture shallow lexical features, while late layers are committed to specific next-token logit projections; intermediate layers capture broad semantic abstractions.
3. **Vector Computation**: The persona vector (c_k) is the mean difference between the trait-expressing activations and non-trait-expressing activations (Δh).
4. **Injection**: At inference, this vector is added to the residual stream: $h^{(l)} \leftarrow h^{(l)} + \alpha c_k$, where α is a continuous steering coefficient defining the strength of the trait[21].

b. Validated Traits and Model Scales

(Strongly Supported): The Persona Vectors methodology has been validated on traits including evil, sycophancy, hallucination, and the standard Big Five dimensions across models like Qwen 2.5-7B and Llama 3.1-8B[21]. Earlier activation steering research, including Contrastive Activation Addition (CAA, Rimsky et al.) and Representation Engineering (RepE, Zou et al.), has successfully steered truthfulness, humor, and politeness across models ranging from 7B to 70B parameters. Inference-Time Intervention (ITI, Li et al.) similarly demonstrated the ability to shift model truthfulness by isolating specific attention heads.

c. Composability and Cross-Trait Entanglement

(Contested): A critical requirement for simulating a 28-facet profile is composability. The linear representation hypothesis suggests that distinct semantic concepts occupy orthogonal directions in high-dimensional space. However, adding 28 simultaneous steering vectors ($\sum_{i=1}^{28} \alpha_i v_i$) often leads to non-linear interference. The literature provides mixed evidence on the orthogonality of highly granular traits (e.g., mathematically distinguishing the latent vector for "Gregariousness" from "Warmth"). Substantial cross-trait entanglement remains an open problem. When steering strength is high across multiple non-orthogonal vectors, the activations saturate, leading to catastrophic degradation of linguistic fluency.

d. Side Effects and the Usable Range

(Strongly Supported): There is a strictly bounded "usable range" for the steering coefficient α. If α is too low, the trait expression is negligible. If α exceeds the empirical threshold, the model's internal representations are pushed out of distribution. This manifests as severe perplexity spikes (the model generates gibberish) and precipitous drops in capability benchmarks like MMLU[21]. Careful layer selection and coefficient scaling are mandatory to maintain conversational coherence.

e. Robustness and Transferability

(Mixed): Vectors extracted on one model family (e.g., Llama) do *not* transfer to another (e.g., Qwen) without complex mapping methodologies, as their latent spaces are geometrically distinct[21]. However, within the same model, steering is remarkably robust across topics and long context windows. Because steering fundamentally biases the activation geometries at every single token generation step, it entirely bypasses the "attention dilution" that plagues long-context prompting[21]. Furthermore, steering scales exceptionally well to larger models (e.g., 70B parameters), which often possess cleaner, more linear representations of complex semantic traits than smaller models.

f. Preventative Steering and Training Data Screening

(Strongly Supported): Persona vectors enable "preventative steering" (CAFT). By projecting the activations of proposed training data onto a known trait vector (e.g., "sycophancy"), engineers can quantify the presence of that trait. This allows for the filtering of misaligned data before fine-tuning, or the active application of a negative steering coefficient during training to suppress undesirable trait shifts[21]. This is highly relevant for multi-agent systems: it provides a mathematical mechanism to screen generative agent trajectory data before utilizing it in an Orca-style self-improvement loop.

4. Head-to-Head Comparison

The following table evaluates the five primary methodologies based on literature consensus across critical deployment metrics for a massive multi-agent system.

Metric	**Prompting**	**LoRA / SFT**	**Full Fine-Tuning**	**P-React (MoE LoRA)**	**Activation Steering**
Setup Effort	Low *(Strong)*	Medium *(Strong)*	High *(Strong)*	Very High *(Strong)*	Medium *(Mixed)*
Compute Cost	Low *(Strong)*	Medium *(Strong)*	Very High *(Strong)*	Medium-High *(Strong)*	Low *(Strong)*
Time to	Minutes	Days	Weeks	Weeks	Hours

First Result	*(Strong)*	*(Strong)*	*(Strong)*	*(Strong)*	*(Strong)*
Fidelity of Trait	Low/Medium *(Strong)*	High *(Strong)*	Very High *(Strong)*	High *(Strong)*	High *(Mixed)*
Continuous Multi-Dim	Mixed *(Contested)*	Binary/Low *(Contested)*	Low *(Contested)*	Binary/Low *(Contested)*	**High (via α)** *(Mixed)*
Drift over Dialogue	High (fades) *(Strong)*	Low *(Strong)*	Low *(Strong)*	Low *(Strong)*	Low *(Strong)*
Capability Cost	High (MMLU drop) *(Strong)*	Low-Medium *(Strong)*	High (Overfitting) *(Strong)*	Low *(Strong)*	High if α > threshold *(Strong)*
Reversibility	Instant *(Strong)*	High (unload) *(Strong)*	None *(Strong)*	High *(Strong)*	Instant *(Strong)*
Per-Agent Custom	Infinite *(Strong)*	Bound by adapters *(Strong)*	N/A *(Strong)*	Bound by N experts *(Strong)*	Infinite vectors *(Contested)*
Scale Serving Cost	High (KV bloat) *(Strong)*	Medium (S-LoRA) *(Mixed)*	Impossible *(Strong)*	Medium-High *(Mixed)*	**Very Low** *(Mixed)*

5. Serving at Scale: Massive Agent Populations

Deploying hundreds to millions of continuous agents fundamentally changes the architectural calculus. Memory bandwidth, VRAM, and the KV cache become the supreme bottlenecks, rendering standard local inference paradigms obsolete.

a. Batching and Serving Engines

(Strongly Supported): Efficient simulation requires advanced serving engines like vLLM or SGLang, which employ continuous batching, PagedAttention, and chunked prefill[24]. Implementing per-request activation steering in continuous batching is deeply complex because sequences are flattened into 1D tensors during the prefill phase.

- **vLLM**: Recent Request for Comments (RFCs), such as the Observation Plugin (RFC 36998) and IBM hooks, propose tensor-level interception inside the forward pass[24]. This allows developers to securely intercept model activations and dynamically inject custom vectors per request in the batch without breaking PyTorch graph optimizations[24].
- **SGLang**: Researchers have developed custom Triton/CUDA operations spliced directly into the decoder layer. Reading activations via standard PyTorch forward hooks forces the engine into eager mode, costing up to +194% Time-Per-Output-Token (TPOT) overhead on vLLM and +76% on SGLang[26]. Custom ops bypass this, allowing activation probes and steering with near-zero overhead while maintaining CUDA graph optimizations[26].

b. Multi-LoRA Serving

(Strongly Supported): To serve multiple archetypes, engines like vLLM utilize custom kernels based on Punica and S-LoRA[19]. These kernels allow multiple LoRA adapters to be applied to different requests within the same batch. Configurations (e.g., enable_lora=True, max_loras, max_lora_rank) dictate total throughput[19]. While highly optimized, physical limitations remain: keeping thousands of concurrent distinct LoRAs in GPU memory is impossible due to VRAM limits. Serving millions of agents requires paging LoRA weights from CPU to GPU dynamically or restricting the system to a smaller set of shared macro-adapters (e.g., 20 foundational archetypes).

c. Cost Comparison per Agent Turn

(Strongly Supported):

- **Prompting**: A 1,000-token persona prompt across 1,000 agents requires storing 1,000 distinct KV caches. This consumes massive VRAM, destroying batch sizes and throughput. While RadixAttention and Prefix Caching (standard in SGLang/vLLM) solve this for shared prompts, injecting unique agent profiles directly into the text fragments the cache, rendering prefix caching virtually useless[26].
- **Adapters**: S-LoRA shares the base model KV cache but requires moving adapter weights into VRAM, incurring a memory bandwidth penalty.
- **Activation Steering**: Extremely cost-efficient. A steering vector is a d_{model} dimension array (e.g., 4096 floats $\approx$ 8KB). Applying this via a custom GPU kernel adds virtually zero memory overhead to the forward pass and preserves exact KV cache prefix sharing up to the point of injection[22].

d. Architecture for Large Populations

(Mixed): The agent-simulation literature recommends a tiered, highly stratified architecture to manage computational load:

1. **Archetypes**: Define 10-20 core demographic/psychological archetypes via LoRA.
2. **Tiered Agents**: "Key players" (agents the user interacts with directly) receive full inference via frontier models (e.g., Llama-3.1-70B) with deep reasoning and graph memory. Background NPCs (the "cheap crowd") utilize highly quantized, smaller models (e.g., Llama-3-8B INT4) with minimal context[13].
3. **Sampling**: Instead of running 1,000,000 distinct active models, the system runs inference

only for currently active agents and probabilistically samples baseline behaviors for the rest using lightweight statistical user-simulation models[13].

6. Hybrids and Recommended Pipeline

No single method fulfills the complex requirements for 28 continuous dimensions, long-term graph memory, strict rule adherence, and massive computational scale.

The Recommended Production Architecture:

1. **Base Model**: Llama-3.1-70B for "key players" and Qwen-2.5-7B for the simulation crowd.
2. **Structural Archetypes (LoRA)**: Train ~20 LoRA adapters representing fundamental worldview archetypes (e.g., "The Introverted Academic," "The Aggressive Rebel"). This permanently alters the fundamental data distribution and syntactic style of the agent without bloated prompts.
3. **Continuous Profile Modulation (Activation Steering)**: Extract 28 Persona Vectors corresponding to the continuous facets. During inference, calculate a composite steering vector for the specific agent based on their numeric profile ($v_{agent} = \sum_{i=1}^{28} \alpha_i c_i$). Apply this composite vector at the residual stream. This handles the *implicit* personality (tone, neuroticism, extraversion) precisely and continuously without wasting a single token of context window.
4. **Explicit Logic (Prompting)**: Use a lightweight, highly structured, prefix-cached system prompt to inject the agent's specific identity, their retrieved episodic memory (from the graph database), and strict if-then behavior rules.

This hybrid approach maximizes VRAM efficiency (allowing for a highly shared prefix cache), preserves base-model reasoning (by minimizing prompt bloat and avoiding MMLU drops), and delivers granular, continuous personality scaling via the activation steering coefficients.

7. Validation Protocol

Validating synthetic personality requires triangulating self-report, behavioral observation, and latent measurement. Relying on a single metric invites deception from the model.

1. **Psychometric Inventories**: Administer standard psychological tests like the IPIP-NEO-120 or the BFI directly to the agents via structured prompting[1]. Target a tolerance threshold of Cronbach's $\alpha \geq 0.85$ and a strong Pearson correlation indicating alignment between the intended continuous score and the measured score.
2. **LLM-as-a-Judge**: Utilize an independent, highly capable model (e.g., GPT-4o) using frameworks like CharacterBox or TRAIT to score generated dialogue[30]. The judge should evaluate adherence to the intended personality, absence of anachronisms, and narrative consistency.
3. **Latent Trait Expression Scores**: Adopt the monitoring methodology of Persona Vectors[16]. Calculate the mathematical projection of the model's generated activations onto the known persona vectors. This ensures the agent is mathematically operating in the intended behavioral region of the latent space, regardless of the generated text[21].
4. **Drift Tolerance and Consistency Tests**: Conduct multi-turn (50+ interactions)

consistency stress tests. If the LLM judge detects a $> 15\%$ deviation from the original profile score over the dialogue, the context window must be forcibly summarized, or the steering vector must be dynamically amplified to pull the model back into alignment.

8. Step-by-Step Prototype and Scaling Plan

Phase 1: Prototype Development

1. **Model Selection**: Start with Qwen/Qwen2.5-7B-Instruct. It demonstrates excellent steerability, possesses strong reasoning baselines, and is fully supported by the persona_vectors pipeline[16].
2. **Vector Extraction**: Fork the safety-research/persona_vectors repository[16]. Utilize Claude-3.7 to generate contrastive prompts for your 28 Big Five facets[21]. Extract the vectors ($c_{1...28}$) by calculating the difference in means at layer 20.
3. **Benchmarking the Composite Vector**: Write a Python script utilizing standard PyTorch register_forward_hook to inject $\sum \alpha_i c_i$.
   - Test orthogonal combinations (e.g., High Extraversion + High Neuroticism).
   - Optimize the α coefficients to stay below the perplexity degradation threshold.
4. **Agent Logic Integration**: Implement a basic script (e.g., using LangChain or LlamaIndex) that fetches a profile and memory graph from a local database, generates a minimal prompt, and runs inference with the custom hook attached.

Phase 2: Scaling to Production

1. **Inference Server Deployment**: Deploy **SGLang** (v0.2.14 or later) to handle continuous batching and maximum throughput[26].
2. **Custom CUDA Hooks**: Because standard PyTorch hooks force the engine into eager mode (ruining throughput), register a custom torch.ops C++/Triton kernel[26]. Inject this directly into SGLang's execution graph. This custom op must read the agent's specific composite steering vector from a pre-allocated tensor (mapped to the batch index) and add it to the residual stream[22].
3. **KV Cache Management**: Utilize SGLang's RadixAttention. Construct the system prompts identically across all agents for the core rules to maximize cache hits. Append the specific agent's memory payload at the very end of the prompt sequence[26].
4. **Hardware Topology**: Deploy on a cluster of H100s or A100s. Route requests via a central gateway (e.g., written in Go) that maps Agent IDs to specific nodes holding the relevant LoRA adapters (via S-LoRA configuration) to maintain optimal cache hit rates and memory locality[27].

Example vLLM Multi-LoRA Configuration (If falling back from SGLang)

```
Python
from vllm import LLM, SamplingParams
from vllm.lora.request import LoRARequest

# Initialize vLLM with Multi-LoRA enabled
llm = LLM(
    model="Qwen/Qwen2.5-7B-Instruct",
    enable_lora=True,
    max_loras=4,  # Maximum concurrent archetypes per batch
    max_lora_rank=32,
    max_cpu_loras=20 # Cache archetypes in CPU RAM
)

# Route requests to specific archetypes
sampling_params = SamplingParams(temperature=0.7, max_tokens=256)
prompts = ["Agent 1 specific prompt", "Agent 2 specific prompt"]

outputs = llm.generate(
    prompts,
    sampling_params,
    lora_request=[
        LoRARequest("introvert_academic", 1, "/path/to/academic/lora"),
        LoRARequest("aggressive_rebel", 2, "/path/to/rebel/lora")
    ]
)
```

9. Open Problems and Unverified Claims

While designing this architecture, several critical constraints within the current literature must be acknowledged:

- **28-Dimensional Composability**: The literature robustly supports single-trait and dual-trait activation steering. However, the claim that 28 distinct, continuous personality vectors can be *simultaneously* injected into the residual stream without severe non-linear interference, catastrophic saturation, or fluency collapse is entirely **unverified** and theoretically contested. High-dimensional vector addition in narrow intermediate layers is highly likely to push activations completely outside the pre-training distribution.
- **Non-Transferability of Vectors**: Persona vectors extracted from Qwen 7B do not reliably map to Llama 8B or 70B. A scalable system requires computationally re-extracting all 28 vectors for every single new model version or architecture deployed.
- **Quantization Impact**: While production models are almost universally quantized (e.g., FP8, INT4) for scale, the impact of low-precision quantization on the high-fidelity geometry of continuous activation steering vectors remains underexplored in the literature.
- **Dynamic Steering Costs**: The precise mathematical scaling laws regarding how the

optimal steering coefficient (α) must shift dynamically as the context window grows significantly longer remain poorly documented. If context grows, the baseline residual stream magnitude shifts, potentially requiring a dynamic, rather than static, α multiplier.

Works cited

1. Personality Traits in Large Language Models - arXiv, https://arxiv.org/pdf/2307.00184
2. Personality Traits in LLMs - Emergent Mind, https://www.emergentmind.com/topics/personality-traits-in-large-language-models
3. PersonaLLM: Investigating the Ability of Large Language Models to, https://arxiv.org/abs/2305.02547
4. Persona Prompting: Make ChatGPT Think Like an Expert (2026), https://prompt-architects.com/blog/45-persona-prompting-make-chatgpt-think-like-an-expert
5. P-Tailor: Customizing Personality Traits for Language Models via, https://arxiv.org/html/2406.12548v1
6. Expert Personas Improve LLM Alignment but Damage Accuracy - arXiv, https://arxiv.org/html/2603.18507v1
7. Stop Telling Your AI It's an Expert: Here's What to Do Instead, https://www.matthewaberham.com/blog/stop-telling-ai-expert-persona-prompting
8. Expert Personas Improve LLM Alignment but Damage Accuracy - arXiv, https://arxiv.org/pdf/2603.18507
9. Playing Pretend: Expert Personas Don't Improve Factual Accuracy, https://gail.wharton.upenn.edu/research-and-insights/playing-pretend-expert-personas/
10. How Personality Traits Shape LLM Risk-Taking Behaviour, https://aclanthology.org/2025.findings-acl.1085.pdf
11. Character-LLM: A Trainable Agent for Role-Playing - alphaXiv, https://www.alphaxiv.org/abs/2310.10158
12. Orca: Enhancing Role-Playing Abilities of Large Language Models, https://tldr.takara.ai/p/2411.10006
13. A Survey on LLM-based Conversational User Simulation - arXiv, https://arxiv.org/html/2604.24977v1
14. Rethinking Role-Playing Evaluation: Anonymous Benchmarking and, https://aclanthology.org/2026.sigdial-1.15.pdf
15. Orca: Enhancing Role-Playing Abilities of Large Language Models, https://arxiv.org/abs/2411.10006
16. Persona Vectors: Monitoring and Controlling Character Traits in, https://github.com/safety-research/persona_vectors
17. Profile-LLM: Dynamic Profile Optimization for Realistic Personality, https://arxiv.org/html/2511.19852v1
18. Large Language Models for Conversational User Simulation, http://ryanrossi.com/pubs/LLM-based_User_Simulated_Data_Generation_Survey.

pdf
19. MultiLoRA Inference - vLLM Documentation, https://docs.vllm.ai/en/v0.6.3/getting_started/examples/multilora_inference.html
20. Persona Vectors: Monitoring and Controlling Character Traits in, https://arxiv.org/abs/2507.21509
21. Persona Vectors: Monitoring and Controlling Character Traits in, https://www.alphaxiv.org/abs/2507.21509
22. Covert Information Transfer through LLM Residual Streams - arXiv, https://arxiv.org/html/2609.27996v1
23. Persona vectors: Monitoring and controlling character traits in, https://medium.com/@danny_54172/persona-vectors-monitoring-and-controlling-character-traits-in-language-models-add537d981cc
24. [RFC]: Observation Plugin for Intercepting & Routing on Activations, https://github.com/vllm-project/vllm/issues/36998
25. Harvesting Activations from a Trillion-Parameter Model - Goodfire AI, https://www.goodfire.com/blog/interpretability-infra-at-frontier-scale
26. Running Safety Probes on Production Inference Servers - Varun Iyer, https://vaiyr.com/writing/probex
27. My AI Lab on an RTX 3090: What Runs, What Broke, and ... - acidbath, https://amenoacids.com/local-ai-lab-architecture/
28. MultiLoRA Inference - vLLM Documentation, https://docs.vllm.ai/en/v0.6.2/getting_started/examples/multilora_inference.html
29. LoRA Adapters - vLLM Documentation, https://docs.vllm.ai/en/latest/features/lora/
30. Profile-LLM - arXiv, https://arxiv.org/pdf/2511.19852
31. CharacterBox: Evaluating the Role-Playing Capabilities of LLMs in, https://aclanthology.org/2025.naacl-long.323.pdf